

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 3.2

**«Создание таблиц базы данных PostgreSQL. Заполнение таблиц
рабочими данными. Резервное копирование и восстановление БД»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Забродин Максим Александрович

Группа К3240

Направление подготовки 09.03.03 Прикладная информатика

Преподаватели: Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург

2026

Цель работы

Овладеть практическими навыками создания таблиц базы данных PostgreSQL, заполнения их рабочими данными, резервного копирования и восстановления БД.

Практическое задание

- Создать базу данных с использованием pgAdmin 4 (согласно индивидуальному заданию).
- Создать схему в составе базы данных.
- Создать таблицы базы данных.
- Установить ограничения на данные: Primary Key, Unique, Check, Foreign Key.
- Заполнить таблицы БД рабочими данными.
- Создать резервную копию БД: с расширением CUSTOM (для восстановления БД) и для листинга в отчёте (формат Plain); при создании резервных копий настроить параметры Dump options для Type of objects и Queries.
- Восстановить БД.

Выполнение

Наименование БД, вариант

Вариант: 9. Предметная область — «Оптовая база».

Оптовая база закупает товары у компаний-поставщиков и поставляет их компаниям-покупателям. Каждый товар имеет производителя; один и тот же товар может поставляться несколькими поставщиками. Поставки и заказы обслуживают сотрудники (менеджеры по поставкам и продажам).

Название создаваемой БД — «Оптовая база».

Схема базы данных, в составе которой созданы таблицы — wholesale.

Инфологическая модель БД

Инфологическая модель предметной области была разработана в лабораторной работе №2 в комбинированной нотации Питера-Чена-Кириллова (рис. 1).

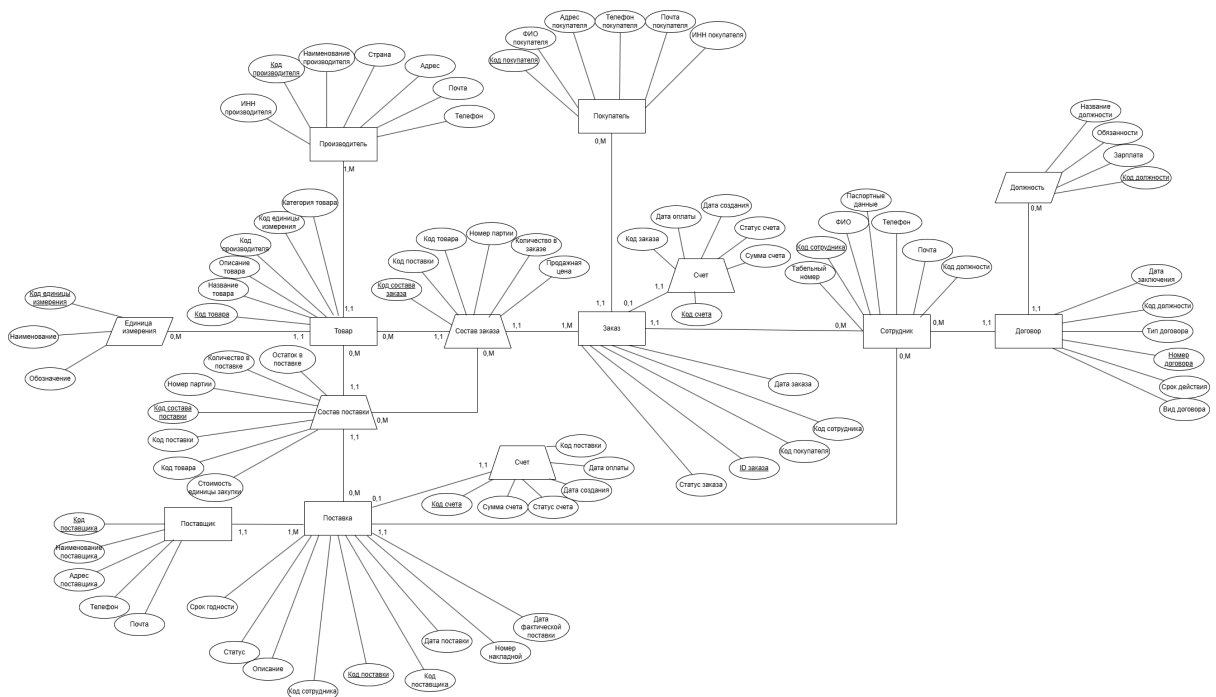


Рисунок 1 — Инфологическая модель БД (нотация Чена-Кириллова), ЛР №2

Логическая модель БД

После создания таблиц, первичных и внешних ключей в PostgreSQL логическая модель базы данных была сгенерирована средствами инструмента Generate ERD в pgAdmin 4 (рис. 2). В соответствии с указанием составные ключи сущностей, использованные в инфологической модели (например, в сущностях «Состав поставки» и «Состав заказа»), заменены суррогатными простыми ключами таблиц (manufacturer_id, product_id, supply_id, order_id и т. д.), генерируемыми автоматически (IDENTITY).

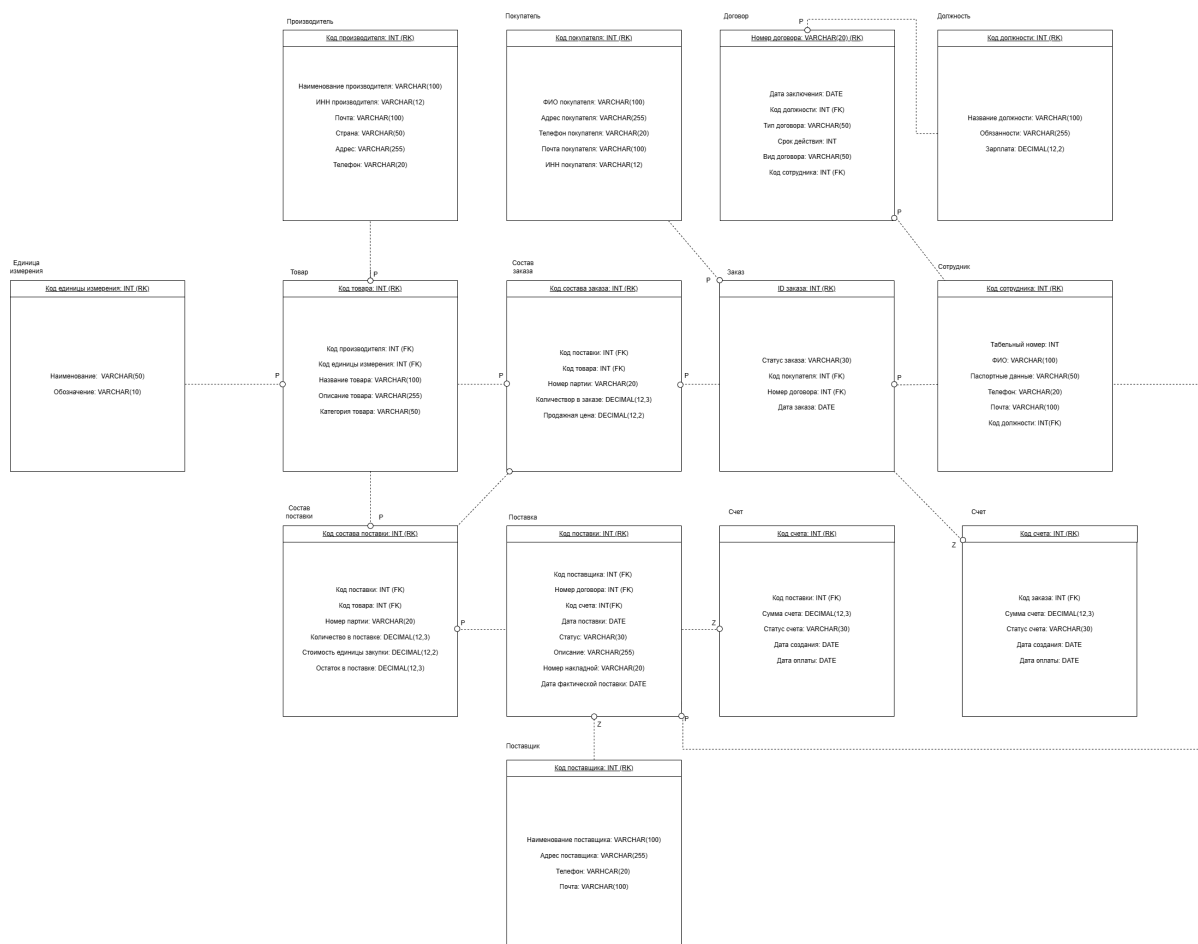


Рисунок 2 — Логическая модель БД, сгенерированная в Generate ERD (pgAdmin 4)

Создание таблиц базы данных и ограничений

В рамках работы в схеме wholesale были созданы следующие таблицы:

- manufacturers — производители товаров;
- customers — покупатели;
- positions — должности сотрудников;
- measurement_units — единицы измерения;
- employees — сотрудники;
- contracts — договоры;
- suppliers — поставщики;
- products — товары;
- supplies — поставки;

- `supply_items` — состав поставки;
- `orders` — заказы;
- `order_items` — состав заказа;
- `supply_invoices` — счета по поставкам;
- `order_invoices` — счета по заказам.

Для всех таблиц первичные ключи реализованы как суррогатные (тип данных `INTEGER GENERATED ALWAYS AS IDENTITY`), кроме таблицы `contracts`, где первичным ключом является номер договора (естественный ключ предметной области).

Для обеспечения целостности данных установлены ограничения:

- `PRIMARY KEY` — на ключевые столбцы каждой таблицы;
- `UNIQUE` — на бизнес-атрибуты, которые не должны повторяться (ИНН, email, телефон, наименование, номер накладной и др.), а также составные уникальные ограничения (например, `uq_products_name_manufacturer`, `uq_supply_items_batch`);
- `CHECK` — на формат телефона и email, диапазоны допустимых значений (количество, цена, сумма, остаток), допустимые статусы и типы (`contract_type`, `contract_kind`, `order_status`, `invoice_status` и т. п.), согласованность дат;
- `FOREIGN KEY` — на все связи между таблицами, с заданием правил `ON UPDATE CASCADE` и `ON DELETE RESTRICT/CASCADE/SET NULL` в зависимости от семантики связи.

Пример фрагмента создания таблицы с ограничениями Primary Key, Unique и Check (manufacturers):

```
CREATE TABLE manufacturers (
    manufacturer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    manufacturer_name VARCHAR(100) NOT NULL UNIQUE,
```

```

manufacturer_inn VARCHAR(12) NOT NULL UNIQUE,
email VARCHAR(100) NOT NULL UNIQUE,
country VARCHAR(50) NOT NULL,
address VARCHAR(255) NOT NULL,
phone VARCHAR(20) NOT NULL UNIQUE,

CONSTRAINT chk_manufacturers_inn
    CHECK (manufacturer_inn ~ '^[0-9]{10}$' OR manufacturer_inn ~ '^[0-9]{12}$'),
CONSTRAINT chk_manufacturers_email
    CHECK (email LIKE '%@%'),
CONSTRAINT chk_manufacturers_phone
    CHECK (phone ~ '^+7\([0-9]{3}\)[0-9]{3}-[0-9]{2}-[0-9]{2}$')
);

```

Листинг 1 — Пример создания таблицы с ограничениями PRIMARY KEY, UNIQUE, CHECK

Пример фрагмента создания внешних ключей (FOREIGN KEY) на таблице supply_items:

```

CONSTRAINT fk_supply_items_supplies
    FOREIGN KEY (supply_id)
    REFERENCES supplies(supply_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,

CONSTRAINT fk_supply_items_products
    FOREIGN KEY (product_id)
    REFERENCES products(product_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT

```

Листинг 2 — Пример ограничений FOREIGN KEY

Заполнение таблиц рабочими данными

Заполнение таблиц выполнено при помощи Query Tool в pgAdmin 4 командами INSERT (скрипт insert_data.sql). После заполнения каждой таблицы корректность данных проверялась запросом SELECT * FROM <имя таблицы>.

Пример заполнения таблицы manufacturers рабочими данными:

```

INSERT INTO manufacturers
    (manufacturer_name, manufacturer_inn, email, country, address, phone)
VALUES

```

```
('TechProm LLC', '7812345678', 'info@techprom.ru', 'Russia',  
'Saint Petersburg, Nevsky Ave, 10', '+7(812)111-22-33'),  
('FoodLine JSC', '7701234567', 'sales@foodline.ru', 'Russia',  
'Moscow, Tverskaya St, 15', '+7(495)222-33-44'),  
('OfficeWorld LLC', '540987654321', 'contact@officeworld.ru', 'Russia',  
'Novosibirsk, Lenina St, 25', '+7(383)333-44-55');
```

Листинг 3 — Пример заполнения таблицы рабочими данными (INSERT)

Аналогичным образом заполнены все остальные таблицы базы данных (customers, positions, measurement_units, employees, contracts, suppliers, products, supplies, supply_items, orders, order_items, supply_invoices, order_invoices) — каждая таблица содержит по 3 строки тестовых данных, согласованных по внешним ключам.

Резервное копирование базы данных

Средствами pgAdmin 4 (пункт контекстного меню «Backup...») созданы две резервные копии базы данных:

- формат Custom (расширение .backup) — для последующего восстановления БД средствами pg_restore; формат сжат по умолчанию и позволяет выбрать, какие объекты восстанавливать;
- формат Plain (расширение .sql) — текстовый SQL-скрипт для включения в листинг отчёта.

При создании обеих резервных копий на вкладке Dump Options были заданы параметры Type of objects и Queries в соответствии с методическими указаниями к работе.

Восстановление базы данных

Восстановление выполнено средствами диалога Restore в pgAdmin 4 (пункт контекстного меню «Restore...» на узле базы данных) с использованием резервной копии формата Custom. Перед восстановлением была создана база данных с тем же именем, как того требует методика восстановления. После запуска восстановления в диалоговом окне Restore было получено сообщение об успешном завершении процесса; структура таблиц, ограничения и данные

были восстановлены в полном объёме, что было подтверждено повторными запросами `SELECT * FROM <имя таблицы>` к восстановленной базе данных.

Дамп, содержащий скрипты работы с БД

Ниже приведены ключевые фрагменты текстового дампа базы данных (формат Plain), полученного при резервном копировании. Дамп содержит команду создания схемы, команды создания таблиц с определением ограничений на значения полей и ограничений на таблицы (первичные и внешние ключи). Полный файл дампа (opt_base_plain.sql) и архив для восстановления (opt_base_custom.backup) прилагаются к отчёту отдельными файлами.

Создание схемы

```
CREATE SCHEMA wholesale;

ALTER SCHEMA wholesale OWNER TO postgres;
```

Листинг 4 — Создание схемы wholesale

Создание таблицы с ограничениями (пример)

```
CREATE TABLE wholesale.customers (
    customer_id integer NOT NULL,
    customer_name character varying(100) NOT NULL,
    address character varying(255) NOT NULL,
    phone character varying(20) NOT NULL,
    email character varying(100) NOT NULL,
    customer_inn character varying(12) NOT NULL,
    CONSTRAINT chk_customers_email CHECK (((email)::text ~ '%@%'::text)),
    CONSTRAINT chk_customers_inn CHECK (((customer_inn)::text ~ '^[\d-]{10}$'::text)
    OR ((customer_inn)::text ~ '^[\d-]{12}$'::text))),
    CONSTRAINT chk_customers_phone CHECK (((phone)::text ~
    '^+7\([\d-]{3}\)[\d-]{3}-[\d-]{2}-[\d-]{2}$'::text))
);
```

Листинг 5 — Пример создания таблицы customers (фрагмент дампа)

Ограничения Primary Key и Unique

```
ALTER TABLE ONLY wholesale.customers
    ADD CONSTRAINT customers_pkey PRIMARY KEY (customer_id);

ALTER TABLE ONLY wholesale.customers
    ADD CONSTRAINT customers_email_key UNIQUE (email);

ALTER TABLE ONLY wholesale.customers
```

```
ADD CONSTRAINT customers_phone_key UNIQUE (phone);

ALTER TABLE ONLY wholesale.customers
ADD CONSTRAINT customers_customer_inn_key UNIQUE (customer_inn);
```

Листинг 6 — Примеры ограничений PRIMARY KEY и UNIQUE (фрагмент дампа)

Ограничения Foreign Key

```
ALTER TABLE ONLY wholesale.supplies
ADD CONSTRAINT fk_supplies_contracts FOREIGN KEY (contract_number)
REFERENCES wholesale.contracts(contract_number)
ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE ONLY wholesale.supplies
ADD CONSTRAINT fk_supplies_supply_invoices FOREIGN KEY (invoice_id)
REFERENCES wholesale.supply_invoices(invoice_id)
ON UPDATE CASCADE ON DELETE SET NULL;

ALTER TABLE ONLY wholesale.order_items
ADD CONSTRAINT fk_order_items_products FOREIGN KEY (product_id)
REFERENCES wholesale.products(product_id)
ON UPDATE CASCADE ON DELETE RESTRICT;
```

Листинг 7 — Примеры ограничений FOREIGN KEY (фрагмент дампа)

Примечание: использованная при резервном копировании настройка Type of objects (Plain-формат) включала выгрузку схемы и ограничений; раздел с данными (команды INSERT/COPY) в текущей версии дампа не выгружен — для полного соответствия требованиям методички (наличие insert-запросов в листинге) рекомендуется пересоздать резервную копию формата Plain с параметром Type of objects = «Schema and data».

Выводы

В ходе выполнения лабораторной работы №3.2 были получены практические навыки создания таблиц базы данных PostgreSQL средствами pgAdmin 4. В схеме wholesale базы данных «Оптовая база» (вариант 9) созданы 14 таблиц, реализующих сущности инфологической модели, разработанной в лабораторной работе №2. Составные ключи сущностей ИЛМ заменены суррогатными простыми ключами. Для всех таблиц установлены ограничения целостности данных: первичные ключи (Primary Key), ограничения уникальности (Unique), проверочные ограничения (Check) и внешние ключи (Foreign Key) с заданием правил ссылочной целостности при обновлении и удалении данных.

Таблицы заполнены рабочими данными с помощью команд INSERT через Query Tool, корректность заполнения проверена запросами SELECT. Выполнено резервное копирование базы данных в форматах Custom (для восстановления) и Plain (для листинга в отчёте), а также произведено восстановление базы данных из резервной копии формата Custom, что подтвердило корректность структуры и сохранность данных.